

Stored Procedures Avançado

Criação, Execução, Parametrização, Otimização, Segurança e Monitoramento de Procedimentos Armazenados no SQL Server.

sp_exemplo.sql

T-SQL

```
-- Estrutura básica de uma Stored Procedure
CREATE PROCEDURE
    @ClienteId INT
AS
BEGIN
    SET NOCOUNT ON
    SELECT
    FROM
    WHERE      @ClienteId
END
GO
```

SSMS · 17.4

✓ Compilada · plano em cache

2

TÓPICOS

24

SLIDES

~60'

DURAÇÃO

Agenda do Módulo

Dois tópicos cobrindo criação, parametrização, otimização, segurança e monitoramento de SPs.

DURAÇÃO ~ 60'

☰ TÓPICO 2.1

01 / 02

Criação, Execução e Parametrização

Da sintaxe básica às opções avançadas (WITH ENCRYPTION, RECOMPILE, EXECUTE AS) e EXECUTE AS) e parametrização estrita (INPUT, OUTPUT, TVP).

- 01 Explicação Teórica Avançada (anatomia + ciclo de vida + opções de criação)
- 02 Parametrização Estrita (Input, OUTPUT, TVP)
- 03 Cenário do Mundo Real (transferência bancária com auditoria)
- 04 Exemplo de Código Prático Completo (TVP + OUTPUT + TRY/CATCH + THROW + XACT_STATE)
- 05 Erros Comuns, Anti-patterns e Desafio

🕒 ~30 MIN

☹ TÓPICO 2.2

02 / 02

Otimização, Segurança e Monitoramento

Parameter Sniffing, Execution Plan Cache, Ownership Chaining, EXECUTE, sp_executesql, sp_executesql, DMVs e Extended Events.

- 01 Parameter Sniffing (causa, efeito, mitigações)
- 02 Reutilização do Plano de Execução (Plan Cache)
- 03 Segurança (Ownership Chaining + sp_executesql anti-injection)
- 04 Monitoramento (sys.dm_exec_procedure_stats + Extended Events)
- 05 Cenário Real, Anti-patterns e Desafio

🕒 ~30 MIN

Total estimado: ~60 minutos · 24 slides · Exemplos práticos em T-SQL

TÓPICO 01 / 02

01

Criação, Execução e Parametrização de Stored Procedures

Da sintaxe básica às opções avançadas (WITH ENCRYPTION, RECOMPILE, EXECUTE AS) e parametrização parametrização estrita com INPUT, OUTPUT e Table-Valued Parameters (TVP).

- 01** → Anatomia de uma SP + ciclo de vida (compile → cache → execute → reuse)
- 02** → Opções avançadas de criação (WITH ENCRYPTION, RECOMPILE, EXECUTE AS)
- 03** → Parametrização estrita: Input, OUTPUT e Table-Valued Parameters (TVP)
- 04** → Cenário real + código completo (TVP + OUTPUT + TRY/CATCH + THROW + XACT_STATE)
- 05** → Erros comuns, anti-patterns e desafio de fixação

ANATOMIA INTERNA

Anatomia de uma SP + Ciclo de Vida

O que acontece dentro do SQL Server quando você executa CREATE PROCEDURE e depois EXEC.

01

PARSE + ALGEBRIZE

Parser valida sintaxe T-SQL. Algebrizer converte para árvore lógica (query tree). Erros de sintaxe param aqui.

02

OPTIMIZE + COMPILE

Query Optimizer gera plano de execução baseado em estatísticas, índices e custos. Plano é compilado em código executável.

03

CACHE PLAN

Plano de execução é armazenado no Plan Cache (memory). Próxima execução pode reutilizá-lo, pulando Parse+Optimize.

04

EXECUTE

Runtime executa o plano cached. Parâmetros reais são substituídos. Resultados retornam ao cliente via TDS.

⚡ Por que SPs são mais rápidas?

A primeira execução paga o custo de parsing + otimização (pode demorar 50-200ms). Execuções subsequentes reutilizam o plano cached (~1-5ms). Em sistemas OLTP de alto volume, a economia compõe-se em dezenas de minutos por hora.

⚠ Quando o cache vira problema?

Se a primeira execução usa um parâmetro atípico (ex: filtro muito seletivo), o plano cached pode ser péssimo para parâmetros típicos. Esse é o fenômeno do Parameter Sniffing — veremos no Tópico 2.2.

✦ OPÇÕES DE CRIAÇÃO

Opções Avançadas de Criação

Três cláusulas que mudam comportamento de segurança, recompilação e contexto de execução.



SINTAXE

WITH ENCRYPTION

OCULTA O CÓDIGO-FONTE

SINTAXE

Criptografa a definição da SP em sys.sql_modules. Impede leitura via sp_helptext ou SELECT definition.

QUANDO USAR

Proteção de PI (propriedade intelectual) em SPs vendor / third-party. Compliance contratual.

QUANDO NÃO USAR

Em SPs internas — dificulta debug, code review e monitoramento (Extended Events não capturam o texto). texto).

```
CREATE PROCEDURE dbo.usp_Critica @Id
INT WITH ENCRYPTION AS BEGIN -- código
invisível -- em sys.sql_modules END
```



SINTAXE

WITH RECOMPILE

RECOMPILA A CADA EXECUÇÃO

SINTAXE

Força recompilação a cada execução. Plano não é cached. Útil quando dados têm distribuição muito assimétrica.

QUANDO USAR

SPs com parâmetros voláteis (ora seletivos, ora amplos) que sofrem parameter sniffing severo.

QUANDO NÃO USAR

Em SPs OLTP de alto volume — custo de recompilação (50-recompilação (50-200ms por execução) acumula.

```
CREATE PROCEDURE dbo.usp_Relatorio
@DataInicial DATE WITH RECOMPILE AS
BEGIN -- recompila a cada EXEC -- não
cacheia plano END
```



SINTAXE

WITH EXECUTE AS

CONTEXTO DE EXECUÇÃO

SINTAXE

Altera o contexto de segurança sob o qual a SP executa. executa. Pode ser CALLER (padrão), OWNER, ou um usuário usuário específico.

QUANDO USAR

Permitir que usuários não-admin acessem tabelas sem ter sem ter SELECT direto — via Ownership Chaining (veremos (veremos no slide 19).

QUANDO NÃO USAR

Para escalar privilégios sem necessidade — vira brecha de brecha de segurança se a SP manipular dados fora do do escopo pretendido.

```
CREATE PROCEDURE dbo.usp_LimpaLog WITH
EXECUTE AS OWNER AS BEGIN -- executa
no contexto -- do owner da SP TRUNCATE
TABLE dbo.Log; END
```

◆ Use WITH ENCRYPTION + EXECUTE AS em conjunto para SPs vendor

PARAMETRIZAÇÃO ESTRITA

Tipos de Parâmetros: Input, OUTPUT e TVP

Três categorias para passar dados para dentro (e fora) da Stored Procedure.

01

Parâmetros de Entrada (Input)

INPUT ONLY

SINTAXE

Declarados com nome + tipo. Recebem valor antes da da EXEC. São read-only dentro da SP (a menos que sejam sejam OUTPUT também).

EXEMPLO

```
CREATE PROCEDURE dbo.usp_ObterCliente
@ClienteId INT,
@Ativo BIT = 1
AS
BEGIN
SELECT * FROM Clientes
WHERE ClienteId = @ClienteId
AND Ativo = @Ativo;
END
```

QUANDO USAR

Sempre que precisar filtrar ou parametrizar — é o caso padrão.

02

Parâmetros de Saída (OUTPUT)

RETORNA VALOR AO CALLER

SINTAXE

Marcados com a cláusula OUTPUT. Permitem que a SP retorne valores escalares além do result set. O caller deve declarar uma variável e passá-la com OUTPUT.

OUTPUT EXEMPLO

```
CREATE PROCEDURE dbo.usp_ObterTotal
@ClienteId INT,
@Total DECIMAL(10,2) OUTPUT
AS
BEGIN
SELECT @Total = SUM(Total)
FROM Vendas
WHERE ClienteId = @ClienteId;
END
-- Caller:
DECLARE @T DECIMAL(10,2);
EXEC dbo.usp_ObterTotal 1024, @T OUTPUT;
PRINT @T;
```

QUANDO USAR

Retornar status code, total acumulado, ID gerado por por SCOPE_IDENTITY(), contagens — sem poluir result set.

03

Table-Valued Parameters (TVP)

LISTA/TABELA COMO PARÂMETRO

SINTAXE

Permitem passar uma tabela inteira como parâmetro. Tipo parâmetro. Tipo é definido previamente via CREATE TYPE TYPE ... AS TABLE. Substitui arrays CSV ou #temp tables.

EXEMPLO

```
-- 1. Define o tipo
CREATE TYPE dbo.ClienteListType
AS TABLE
( ClienteId INT PRIMARY KEY );
-- 2. SP recebe o TVP (READONLY)
CREATE PROCEDURE dbo.usp_Processar
@Clientes dbo.ClienteListType READONLY
AS
BEGIN
SELECT c.* FROM @Clientes k
JOIN Clientes c ON c.ClienteId=k.ClienteId;
END
```

QUANDO USAR

Passar listas de IDs (bulk operations), upload em massa, massa, filtros com N valores dinâmicos. TVPs são READONLY dentro da SP.

⚠ TVPs são READONLY — não dá para fazer INSERT/UPDATE/DELETE na @TVP dentro da SP

 CENÁRIO REAL

Transferência Bancária com Auditoria

Por que SP é a única implementação correta para esse caso.

O PROBLEMA DE NEGÓCIO

Uma fintech precisa implementar transferência entre contas correntes. A operação envolve: (1) validar que a conta validar que a conta origem tem saldo suficiente; (2) debitar da origem; (3) creditar no destino; (4) registrar a registrar a transação em tabela de auditoria; (5) rejeitar a operação se qualquer passo falhar — garantindo garantindo atomicidade ACID.

REQUISITOS TÉCNICOS

- **Atomicidade ACID:** ou todos os passos acontecem, ou nenhum. Sem transferência parcial.
- **Auditoria obrigatória:** registrar usuário, data, valores, contas envolvidas — mesmo em caso de erro.
- **Validação explícita:** saldo insuficiente deve disparar erro com mensagem clara (não silencioso).
- **Tratamento de erro estruturado:** usar TRY/CATCH, não checagens IF @@ERROR aninhadas (legado).
- **Idempotência:** cliente recebe um status (@Sucesso BIT OUTPUT) — pode retentar com segurança.
- **Log estruturado:** registrar erros em tabela própria com ERROR_NUMBER, ERROR_LINE, ERROR_PROCEDURE.

Por que Stored Procedure?

3 RAZÕES TÉCNICAS

- 01 Atomicidade transacional:** BEGIN TRAN/COMMIT dentro da SP garante que débito+crédito+auditoria sejam atômicos. Aplicação não consegue replicar isso com 3 chamadas separadas.
- 02 Plano cached:** a SP é compilada uma vez. Transferências em massa (1000+ por segundo) não pagam custo de parse a cada EXEC.
- 03 Segurança:** aplicação só recebe EXECUTE na SP — não ganha SELECT/UPDATE direto em Contas. Reduz superfície de ataque.

Sem SP, a aplicação precisaria de 4 statements SQL separados em 4 round-trips, sem garantia atômica entre eles.

★ CÓDIGO PRÁTICO — PARTE 1/2

Estrutura Base da SP

Assinatura da procedure `usp_TransferirFundos` — com TVP (lista de auditoria) e parâmetro OUTPUT de status.

PONTOS-CHAVE

- O TVP `@AuditInfo` carrega dados de contexto (usuário, (usuário, IP, sessão).
- `@Sucesso` BIT OUTPUT retorna status para o cliente.
- `SET NOCOUNT ON` evita `DONE_IN_PROC` (reduce network traffic).
- `SET XACT_ABORT ON` garante rollback automático em automático em erro grave.
- Validação de saldo via `IF EXISTS` — explícita, não depende de constraint.
- Estrutura prepara para o bloco `TRY/CATCH` do próximo slide.

usp_TransferirFundos.sql

TVP + OUTPUT

```
-- =====
-- SP: usp_TransferirFundos
-- Autor: Banco de Dados · Módulo 2
-- Descrição: transferência atômica com TVP e OUTPUT
-- =====

CREATE OR ALTER PROCEDURE
    @DeConta      INT
    @ParaConta     INT
    @Valor         DECIMAL 10 2
    @AuditInfo     BIT OUTPUT          READONLY  -- TVP
    @Sucesso       BIT OUTPUT
    @Mensagem      VARCHAR 200 OUTPUT

AS
BEGIN
    SET NOCOUNT ON
    SET XACT_ABORT ON  -- rollback automático em erro grave
    SET @Sucesso 0
    SET @Mensagem

    -- Próximo slide: bloco TRY/CATCH completo
    -- com BEGIN TRAN, validação, INSERT em auditoria, etc.
END
GO
```

Compilada · plano cached

OUTPUT: @Sucesso, @Mensagem

⚠ Requer `CREATE TYPE dbo.AuditContextType AS TABLE (ClienteId INT, Usuario VARCHAR(50), IPAddress VARCHAR(45))`

✦ CÓDIGO PRÁTICO — PARTE 2/2

TRY/CATCH + THROW + XACT_STATE

Continuação da SP `usp_TransferirFundos`
`usp_TransferirFundos` — bloco
transacional completo.
PADRÃO ROBUSTO

- **TRY** envolve toda operação transacional.
- **IF EXISTS** valida saldo antes de debit (fail fast).
- **THROW 50000** dispara erro customizado customizado com mensagem clara.
- **INSERT em LogErros** captura ERROR_NUMBER/SEVERITY/MESSAGE/PROCEDURE/LINE.
- **XACT_STATE() <> 0** só faz rollback se houver transação ativa.
- **THROW final** (sem args) re-lança erro original para o cliente.

usp TransferirFundos.sql

- ✓ Tratamento ACID completo

( TRY/CATCH · ACID)

6 FUNÇÕES DE ERRO + THROW

PARTE 2/2 · COMPLETO

 ANTI-PATTERNS

Erros Comuns e Anti-patterns

O que desenvolvedores costumam errar ao criar SPs — e como corrigir.



01 PROBLEMA

Sem SET NOCOUNT ON

A cada statement, SQL Server envia mensagem DONE_IN_PROC ao cliente. Em SPs com 100 statements, são 100 mensagens trafegando — sobrecarrega rede e cliente.

CORREÇÃO

Sempre `SET NOCOUNT ON` como primeira linha do BEGIN. Em C# (.NET), reduz tráfego e elimina need NextResult() desnecessário.



02 PROBLEMA

Sem TRY/CATCH

SPs legadas usam `IF @@ERROR <> 0` após cada statement. @@ERROR é resetado pelo próximo — fácil de perder erros.

CORREÇÃO

Estrutura moderna: `BEGIN TRY ... END TRY BEGIN CATCH ... END CATCH`. Use THROW para re-lançar e ERROR_* functions para log.



03 PROBLEMA

Confundir GO com T-SQL

GO é separador de batch reconhecido apenas por SSMS/sqlcmd. Tentar executar GO via SqlCommand ou JDBC ou JDBC falha silenciosamente.

CORREÇÃO

GO não é instrução T-SQL. Em código cliente, envie batches separados via SqlCommand separados, ou use sp_executesql para batch único.



04 PROBLEMA

Usar RETURN para dados

RETURN só retorna INT (status code). Devs retornam IDs, IDs, códigos, contagens como INT — perde tipo.

CORREÇÃO

Para retornar dados escalares, use parâmetro OUTPUT tipado (INT, VARCHAR, DATETIME). Reserve RETURN para status code (0=ok, non-zero=erro).



05 PROBLEMA

SQL dinâmico concatenado

`SET @sql = 'SELECT * FROM Clientes WHERE Nome = ''' + @nome + '''` — vulnerável a SQL Injection, causa recompilação a cada execução.

CORREÇÃO

Use sp_executesql com parâmetros: `EXEC sp_executesql @sql, N'@nome VARCHAR(50)', @nome`. Plano cached, sem injection.



06 PROBLEMA

SP sem documentação

SPs criadas sem comentários, sem schema da assinatura, assinatura, sem tipo de retorno — viram black box para manutenção.

CORREÇÃO

Cabeçalho padrão no topo: autor, data, descrição, parâmetros (nome/tipo/direção/descrição), exemplo de uso, dependências.

✓ Padrão moderno: SET NOCOUNT ON + TRY/CATCH + THROW + sp_executesql

◆ DESAFIO · TÓPICO 2.1

Desafio: SP de Registro de Pedidos

Crie uma Stored Procedure completa aplicando todos os conceitos vistos.

ENUNCIADO

Implemente a SP `dbo.usp_RegistrarPedido` que recebe: (a) um `ClienteId` INT; (b) um TVP contendo itens (`ProdutoId`, `Quantidade`); (c) um parâmetro `@PedidoId` INT OUTPUT para retornar o ID gerado. A que o cliente existe e está ativo; validar que todos os produtos existem e têm estoque; inserir o pedido em `dbo.Pedidos` (com `SYSDATETIME`); inserir os itens em `dbo.PedidosItens`; baixar estoque em `dbo.Produtos`; registrar auditoria; retornar `@PedidoId` e `@Sucesso`. Tudo em transação atômica com TRY/CATCH + TRY/CATCH + THROW.

DICAS

- Crie um TYPE `dbo.PedidoItensType` AS TABLE (`ProdutoId` INT, `Quantidade` INT) PRIMARY KEY (`ProdutoId`).
- Use `SCOPE_IDENTITY()` após INSERT em Pedidos para capturar o novo `PedidoId`.
- Verifique estoque com `IF EXISTS (... WHERE NOT EXISTS (SELECT 1 FROM dbo.Produtos p JOIN @Itens i ON p.ProdutoId = i.ProdutoId WHERE p.Estoque < i.Quantidade))`.
- Estrutura: `BEGIN TRY → BEGIN TRAN → validações → INSERTs → UPDATE → COMMIT. BEGIN CATCH → rollback + log + THROW.`

CrITÉRIOS de Avaliação

CHECKLIST · 7 ITENS

- ✓ **01**—TVP criado e passado corretamente
- ✓ **02**— Parâmetro `@PedidoId` OUTPUT funciona
- ✓ **03**—Validação de cliente ativo
- ✓ **04**—Validação de estoque para todos os itens
- ✓ **05**—Transação atômica (`BEGIN TRAN/COMMIT`)
- ✓ **06**— TRY/CATCH + THROW + `XACT_STATE`
- ✓ **07**—Auditoria em tabela de log

TÓPICO 02 / 02

02

Otimização, Segurança e Monitoramento de Stored Procedures

Parameter Sniffing, Execution Plan Cache, Ownership Chaining, permissão EXECUTE, sp_executesql anti-SQL-injection, DMVs de diagnóstico e Extended Events.

- 01 → Parameter Sniffing (causa, efeito, mitigações: OPTIMIZE FOR, RECOMPILE, variável local)
- 02 → Reutilização do Plano de Execução (Plan Cache) e custo de recompilação
- 03 → Segurança: Ownership Chaining + permissão EXECUTE + sp_executesql anti-injection
- 04 → Monitoramento: sys.dm_exec_procedure_stats + Extended Events
- 05 → Cenário real, anti-patterns e desafio de fixação

◆ PARAMETER SNIFFING

Parameter Sniffing: Causa, Efeito e Mitigação

O problema nº 1 de performance em Stored Procedures parametrizadas.

CAUSA

Primeira execução atípica

Na primeira EXEC de uma SP, o Query Optimizer gera o plano usando o valor concreto do parâmetro parâmetro recebido. Se esse valor é atípico (ex: filtro extremamente seletivo), o plano é gerado para para aquele caso específico — e cached.



EFEITO

Plano cached péssimo para outros params

Próximas EXECs reutilizam o plano cached. Se um parâmetro comum (ex: filtro amplo) é passado, o plano otimizado para o caso seletivo força um Index Seek + Key Lookup em vez de Table Scan — resultado: SP que deveria rodar em 50ms demora 30s.



MITIGAÇÃO

OPTIMIZE FOR, RECOMPILE, variável local

Três técnicas principais: (1) `OPTION (OPTIMIZE FOR (@param UNKNOWN))` — usa densidade estatística; (2) `OPTION (RECOMPILE)` — recompila a cada EXEC; (3) declarar variável local `DECLARE @local = @param` — desativa sniffing. Veremos cada uma no próximo slide.

Exemplo clássico: SP de relatório por data

Um cliente chama `EXEC usp_RelatorioVendas @DataInicial = '1900-01-01'` na primeira execução (filtro amplo, 5M linhas, plano = Table Scan). O plano cached Table Scan. Quando o cliente chama `@DataInicial = '2026-09-01'` (filtro seletivo, 50 linhas), o Table Scan ainda é usado — deveria ser Index Seek. Resultado: relatório que deveria rodar em 80ms demora 8s.

MITIGAÇÕES

Mitigando Parameter Sniffing

Três técnicas, três trade-offs — escolha conforme o cenário.



OPÇÃO 01

OPTIMIZE FOR

DICA DE OTIMIZAÇÃO

SINTAXE

```
CREATE PROCEDURE .
    @DataInicialDATE
AS
BEGIN
    SELECT * FROM
        WHERE
            >=@DataInicial
    OPTION (OPTIMIZE FOR
        (@DataInicialUNKNOWN) );
END
```

COMO FUNCIONA

OPTIMIZE FOR UNKNOWN usa o histograma médio das médio das estatísticas (density vector) em vez do valor real valor real do parâmetro. Gera um plano médio — funciona funciona bem para distribuições moderadamente assimétricas.

PROS

- ✓ Mantém plano cached (sem recompilar)
- ✓ Funciona para a maioria dos casos

CONTRAS

- ⚠ Não resolve casos extremos de skew
- ⚠ Requer SQL Server 2008+ (OPTION RECOMPILE unknown em versões antigas)



OPÇÃO 02

OPTION (RECOMPILE)

RECOMPILA A CADA EXECUÇÃO

SINTAXE

```
CREATE PROCEDURE .
    @DataInicialDATE
AS
BEGIN
    SELECT * FROM
        WHERE
            >=@DataInicial
    OPTION (RECOMPILE);
END
```

COMO FUNCIONA

Recompila a SP a cada execução — não cached o plano. O plano. O Optimizer usa o valor real do parâmetro sempre. sempre. Garante plano ótimo para cada caso.

PRÓS

- ✓ Plano sempre ótimo para o valor atual
- ✓ Resolve skew extremo

CONTRAS

- ⚠ Custo de recompilação (50-200ms/exec)
- ⚠ Não recomendado para OLTP de alto volume



OPÇÃO 03

Variável Local

DESATIVA O SNIFFING

SINTAXE

```
CREATE PROCEDURE .
    @DataInicialDATE
AS
BEGIN
    DECLARE @DataLocalDATE;
    SET @DataLocal=@DataInicial;
    SELECT * FROM
        WHERE
            >=@DataLocal
END
```

COMO FUNCIONA

Quando o Optimizer vê uma variável local (não um parâmetro), ele NÃO cheira o valor — usa a density vector vector (estimativa média). Equivalente a OPTIMIZE FOR FOR UNKNOWN, mas aplicado em nível de variável. variável.

PROS

- ✓ Sintaxe simples, sem hints
- ✓ Compatível com versões antigas do SQL Server

CONTRAS

- ⚠ Requer declaração extra
- ⚠ Menos óbvio para futuros desenvolvedores (intenção intenção não-documentada)

Boa prática: OPTIMIZE FOR UNKNOWN como padrão; RECOMPILE só em casos extremos

 PLAN CACHE

Reutilização do Plano de Execução

Como o SQL Server cached e reusa planos — e quando isso vira problema.

O QUE É O PLAN CACHE

Plan Cache é uma região do SQL Server memory (dentro do Buffer Pool) que armazena planos de execução de execução compilados. Quando uma SP é executada pela primeira vez, o plano é gerado e cached. Proximas EXECs pulam parsing/otimização e reusam o plano cached — economia de 50-200ms por 50-200ms por execução.

CUSTO DE RECOMPILAÇÃO

- Parse + Algebrize: ~5-20ms — valida sintaxe, gera query tree.
- Optimize: ~30-150ms — gera plano baseado em estatísticas e custos.
- Compile: ~10-30ms — traduz plano em código executável.
- Total: 45-200ms por compilação. Em OLTP de alto volume, reuso = economia massiva.

QUANDO O PLANO É REJEITADO

- Mudança de schema (ALTER TABLE, novo índice) invalida planos dependentes.
- Estatísticas atualizadas com grande mudança de distribuição.
- DBCC FREEPROCCACHE limpa manualmente (apenas dev/test).
- SQL Server restart (limpa todo o cache).

COMO MONITORAR

Use `sys.dm_exec_cached_plans` para ver planos cached, `sys.dm_exec_procedure_stats` para estatísticas por SP, `sys.dm_exec_sql_text` para o texto SQL. Veremos essas DMVs no slide 21.

Query de diagnóstico

TOP 10 SPS MAIS EXECUTADAS

● ● ● diagnóstico.sql

Saída: nome da SP, quantas vezes executada, tempo médio, CPU total, e plano XML.

◆ Dica: nunca DBCC FREEPROCCACHE em produção – recompila todas as SPs ao mesmo tempo

CENÁRIO REAL

Relatório que Ora Voa, Ora Arrasta

Um caso clássico de parameter sniffing em produção.

O PROBLEMA EM PRODUÇÃO

Um e-commerce usa a SP `dbo.usp_RelatorioVendas` para um dashboard de vendas. Há duas telas chamam: (a) tela diária do gerente — filtra por `@DataInicial = hoje` (~5k linhas, roda em 80ms); (b) anual do CFO — filtra por `@DataInicial = 2026-01-01` (~5M linhas, roda em 8s). Tudo funciona por meses até...

O INCIDENTE

Numa manhã de segunda-feira, o CFO abre o dashboard anual às 7h. Resultado: a SP demora 8s — esperado. Mas às 9h, o gerente tenta o dashboard diário e a SP também demora 8s (em vez dos 80ms esperados). Em pânico, o DBA recebe o ticket. Reiniciar o serviço resolve temporariamente — mas na terça-feira o problema volta.

SINTOMAS OBSERVADOS

- SP rodando em 8s para filtro diário (deveria ser 80ms).
- Plano de execução atual mostra **Clustered Index Scan** em Vendas — deveria ser Index Seek.
- `sys.dm_exec_procedure_stats` mostra mesma SP com alto `last_elapsed_time`.
- Reiniciar serviço resolve — mas é só até a próxima execução com filtro atípico.
- DBA suspeita de 'corrupção de índice' — mas DBCC CHECKDB não reporta erros.

Diagnóstico Correto

NÃO É CORRUPÇÃO — É PARAMETER SNIFFING

O CFO executou primeiro (filtro anual amplo) — o optimizer gerou plano ótimo para 5M linhas (Clustered Index Scan). Esse plano foi cached. Quando o gerente executou depois (filtro diário seletivo), o plano cached foi reusado — mas Scan em 5M linhas para 5k linhas é péssimo. Deveria ter gerado Index Seek para 5k linhas.

COMO CONFIRMAR

- Comparar plano atual com plano estimado para o parâmetro diário.
- Verificar `sys.dm_exec_procedure_stats` para ver quando o plano foi cached.
- Capturar plano estimado via `SET SHOWPLAN_XML ON` para o filtro diário.
- Se o plano estimado (sem cache) for diferente do plano cached — confirmado param sniffing.

PRÓXIMOS PASSOS

Aplicar uma das mitigações do slide 14 — veremos a solução no próximo slide.

⚠ Sintoma clássico: a mesma SP é rápida e lenta dependendo da ordem de chamada

ANTES · PROBLEMÁTICO

SP Vulnerável a Param Sniffing

Sem nenhuma mitigação — primeira execução execução dita o plano para sempre.

O QUE ESTÁ ERRADO

- Parâmetro @DataInicial é sniffado na primeira execução.
- Sem OPTION hint — optimizer usa valor real valor real do parâmetro.
- Plano cached para sempre (ou até invalidação).
- Se primeira execução tem filtro amplo, plano = Scan péssimo para filtro seletivo.
- Resultado: SP 'ora rápida, ora lenta' dependendo da ordem.

usp_RelatorioVendas_antes.sql

ANTES · VULNERÁVEL

```
-- =====
-- SP: usp_RelatorioVendas (VERSÃO ANTES - PROBLEMÁTICA)
-- Sintoma: ora 80ms, ora 8s, dependendo de quem chama primeiro
-- =====
CREATE OR ALTER PROCEDURE
    @DataInicial DATE
    @ClienteId INT NULL
AS
BEGIN
    SET NOCOUNT ON
    -- Nenhum hint aplicado - optimizer snifa @DataInicial
    -- na primeira execução e cached o plano para sempre.
    SELECT

        AS
    FROM
    INNER JOIN
    ON
    WHERE
        @DataInicial
        AND @ClienteId IS NULL
        OR
        @ClienteId
    ORDER BY
        DESC
END
GO
-- Exemplo do incidente (na primeira execução do dia):
-- EXEC dbo.usp_RelatorioVendas @DataInicial = '2026-01-01';
-- Plano cached ótimo para 5M linhas (Scan).
-- Mas a chamada subsequente com @DataInicial = '2026-09-12'
-- ainda usa o Scan - 8s em vez de 80ms.
```

⚠ Performance variável — depende da ordem de chamada

8s vs 80ms

⚠ Sintoma clássico de parameter sniffing — SP funciona bem até um dia não funciona mais

DEPOIS · OTIMIZADO

SP com Mitigações Aplicadas

Três versões equivalentes — escolha conforme conforme o perfil de uso.

O QUE MUDOU

- ✓ **Versão A:** OPTION (OPTIMIZE FOR UNKNOWN) — usa densidade estatística estatística.
- ✓ **Versão B:** OPTION (RECOMPILE) — recompila a cada EXEC.
- ✓ **Versão C:** Variável local — desativa sniffing implicitamente.
- ✓ Catch-all (@param IS NULL OR ...) deve ser deve ser substituído por IF lógico (versão C (versão C mostra como).
- ✓ Em todas, sys.dm_exec_procedure_stats mostra performance estável.

usp_RelatorioVendas_depois.sql

```
-- =====
-- VERSÃO A: OPTIMIZE FOR UNKNOWN (recomendado p/ maioria)
-- Plano médio cached — funciona bem em distribuições moderadas
-- =====
CREATE OR ALTER PROCEDURE
    @DataInicial DATE
    @ClienteId INT NULL
AS
BEGIN
    SET NOCOUNT ON
    SELECT
        FROM
        INNER JOIN
        WHERE
            @DataInicial
            AND @ClienteId IS NULL OR
            ORDER BY
                DESC
            @ClienteId
    OPTION OPTIMIZE FOR @DataInicial UNKNOWN @ClienteId
END
GO
-- =====
-- VERSÃO B: OPTION (RECOMPILE) (skew extremo)
-- =====
CREATE OR ALTER PROCEDURE
    @DataInicial DATE
    @ClienteId INT NULL
AS
BEGIN
    SET NOCOUNT ON
    SELECT
        FROM
        INNER JOIN
        WHERE
            @DataInicial
            AND @ClienteId IS NULL OR
            ORDER BY
                DESC
            @ClienteId
    OPTION RECOMPILE
END
GO
-- =====
-- VERSÃO C: Variável local + IF lógico (legado)
-- Desativa sniffing implicitamente; quebra catch-all em branches
-- =====
CREATE OR ALTER PROCEDURE
    @DataInicial DATE
    @ClienteId INT NULL
AS
BEGIN
    SET NOCOUNT ON
    DECLARE @DataLocal DATE @DataInicial
    DECLARE @ClienteLocal INT @ClienteId
    IF @ClienteLocal IS NULL
    SELECT
        FROM
        INNER JOIN
        WHERE
            @DataLocal
            ORDER BY
                DESC
        AS
    ELSE
    SELECT
        FROM
        INNER JOIN
        WHERE
            @DataLocal
            AND
            @ClienteLocal
            ORDER BY
                DESC
        AS
END
GO
```

OWNERSHIP CHAIN

Ownership Chaining + Permissão EXECUTE

Como permitir que usuários acessem dados via SP — sem conceder SELECT direto.

CONCEITO

Ownership Chaining (Cadeia de Propriedade) é quando objetos referenciados dentro de uma SP pertencem a uma SP pertencem ao mesmo owner da SP. Nesse caso, SQL Server **NÃO** verifica permissões em cada objeto em cada objeto — confia no fato de que o owner já validou.

CENÁRIO TÍPICO

👤 **Usuário app_user** | Tem permissão EXECUTE em dbo.usp_LerClientes; NÃO tem SELECT direto.



📊 **SP dbo.usp_LerClientes** | Owner: dbo (schema owner). Faz SELECT em dbo.Clientes.



📊 **Tabela dbo.Clientes** | Owner: dbo. Mesmo owner da SP → ownership chain válido.

Como app_user → SP (EXECUTE ok) → Tabela (mesmo owner) → acesso liberado sem SELECT direto.

🛡️ Princípio do menor privilégio: app ganha EXECUTE, não SELECT

Exemplo Prático

PADRÃO DEFENSIVO

🔴🟡🟢 ownership_chain.sql

```
-- 1. Cria schema dedicado para SPs
CREATE SCHEMA
GO

-- 2. SP criada no schema app, owner = dbo
CREATE PROCEDURE
    @Ativo BIT 1
    WITH EXECUTE AS OWNER
AS
BEGIN
    SET NOCOUNT ON
    SELECT
    FROM
    WHERE @Ativo

END
GO

-- 3. Concede apenas EXECUTE (sem SELECT!)
GRANT EXECUTE ON
    TO
-- app_user NÃO precisa de SELECT em dbo.Clientes
-- Ownership chain valida acesso via owner da SP
```

Cuidado: se SP em schema diferente do owner da tabela, ownership chain quebra — SQL Server verifica permissão em cada objeto. cada objeto. Solução: **EXECUTE AS OWNER.**

SQL DINÂMICO SEGURO

sp_executesql parametrizado — anti-SQL-injection

Como executar SQL dinâmico dentro de SPs sem abrir brecha para injeção.

ANTES · CONCATENADO

EXEC(@SQL)

● ● ● sp_danger.sql

VULNERÁVEL

```
-- ✗ VULNERÁVEL a SQL Injection
CREATE PROCEDURE
    @Nome NVARCHAR 100
AS
BEGIN
    DECLARE @sql NVARCHAR MAX
    SET @sql N'SELECT * FROM dbo.Clientes WHERE Nome LIKE ''%'
        @Nome '%''
    EXEC
        @sql
    -- Atacante passa: @Nome = ''; DROP TABLE Clientes;--''
    -- Resultado: SELECT * ...; DROP TABLE ...
END
GO
```

⚠ SQL Injection · sem cache de plano

RISCO CRÍTICO

DEPOIS · PARAMETRIZADO

SP_EXECUTESQL @SQL, @PARAMS,
@P

● ● ● sp_seguro.sql

SEGURO · CACHED

```
-- ✓ SEGURO - SQL Injection impossível
CREATE PROCEDURE
    @Nome NVARCHAR 100
AS
BEGIN
    DECLARE @sql NVARCHAR MAX
    SET @sql N'SELECT * FROM dbo.Clientes WHERE Nome LIKE @pattern'
    DECLARE @pattern NVARCHAR 102
    SET @pattern N'%' @Nome N'%'
    EXEC
        @sql
        N'@pattern NVARCHAR(102)'
        @pattern
    -- Atacante passa: @Nome = ''; DROP TABLE ...
    -- Resultado: busca literal - DROP TABLE nunca executa
    -- Bônus: plano cached - reusado em próxima execução
END
GO
```

✓ Injection-proof · plano cached

PRODUÇÃO-READY

✓ sp_executesql separa SQL de parâmetros — DB engine trata parâmetros como dados, não código

} MONITORAMENTO

Monitorando SPs com DMVs e Extended Events

As ferramentas oficiais para diagnóstico de performance de SPs em produção.

REFERÊNCIA · 8 DMVS FUNDAMENTAIS

DMV / Função	O que mostra	Quando usar
<code>sys.dm_exec_procedure_stats</code>	Estatísticas de execução por SP: <code>execution_count</code> , <code>last/total elapsed_time</code> , <code>total_worker_time</code> , <code>last_logical_reads</code>	Performance agregada por SP — top consumers
<code>sys.dm_exec_sql_text(plan_handle)</code>	Texto SQL do plano (útil em CROSS APPLY com <code>procedure_stats</code>)	Identificar qual statement dentro da SP consome mais
<code>sys.dm_exec_query_plan(plan_handle)</code>	Plano de execução em XML (mostra operadores, custos)	Diagnosticar scans vs seeks, parâmetros sniffed
<code>sys.dm_exec_cached_plans</code>	Planos cached no server — <code>use_counts</code> , <code>size_in_bytes</code> , <code>objtype</code>	Ver cache total e identificar planos ineficientes
<code>sys.dm_exec_requests</code>	Execuções em andamento — para sessões ativas	Ver o que está rodando AGORA (long-running queries)
<code>sys.dm_os_waiting_tasks</code>	Quem está esperando e por quê (<code>wait_type</code>)	Diagnosticar locks, latches, blocks
<code>sys.dm_exec_query_stats</code>	Estatísticas por query (statement level, não SP)	Drill-down dentro de uma SP com vários SELECTs
<code>sys.dm_io_virtual_file_stats</code>	I/O por database file (ler/escrita física)	Identificar se SP é I/O-bound vs CPU-bound

◆ DICA: Query Store (SQL 2016+) é alternativa mais visual — captura histórico de regress

Extended Events — alternativa leve

CAPTURA GRANULAR

Extended Events (XEEvents) é o sucessor do SQL Profiler (deprecated). Captura eventos como `sp_statement_completed`, `sp_statement_completed`, `module_end`, `error_reported` — sem — sem o overhead do Profiler.

●●● capture_sps.sql

```
CREATE EVENT SESSION
ON SERVER
ADD EVENT
    ACTION

    WHERE

        'usp_RelatorioVendas'

ADD TARGET
    SET
        'C:\temp\sps.xel'
WITH
    40
    ON

GO
ALTER EVENT SESSION
ON SERVER STATE START
GO
```

Analisar via SSMS: Object Explorer → Management → Extended Extended Events → [session] → Watch Live Data.

⚠ ANTI-PATTERNS · TÓPICO 2.2

Anti-patterns de Otimização e Segurança

Os 6 erros mais comuns que DBAs e devs cometem em SPs em produção.



PROBLEMA · 01

WITH RECOMPILE em tudo

Dev coloca `WITH RECOMPILE` em todas as SPs por 'precaução' — recompilação desnecessária CPU.

CORREÇÃO

Aplicar RECOMPILE só em SPs com skew extremo confirmado via `sys.dm_exec_procedure_stats`. Caso geral: `OPTIMIZE FOR UNKNOWN`.



PROBLEMA · 02

GRANT SELECT direto em tabelas

Para 'facilitar', DBA concede SELECT em `dbo.Clientes` para `dbo.Clientes` para `app_user` — abre superfície de ataque, ataque, burla ownership chain.

CORREÇÃO

Conceder apenas EXECUTE em SPs; usar `EXECUTE AS OWNER` para chains que cruzam schemas. SELECT direto só para roles de DBA.



PROBLEMA · 03

EXEC() em vez de sp_executesql

Usar `EXEC(@sql)` em vez de `sp_executesql` — aceita apenas string, não cached plano, vulnerável a injection se concatenado.

CORREÇÃO

`sp_executesql` com parâmetros explícitos: `sp_executesql @sql, N'@p1 INT', @val`. cached + injection-proof.



PROBLEMA · 04

DBCC FREEPROCCACHE em produção

Para 'resolver lentidão', Dev roda `DBCC FREEPROCCACHE` em produção — limpa TODOS os planos cached, todas as SPs recompilam simultaneamente.

CORREÇÃO

Identificar plano problemático via `plan_handle` e usar `DBCC FREEPLAN(plan_handle)` — remove só aquele. Em último caso, `ALTER DATABASE SCOPED CONFIGURATION CLEAR PROCEDURE_CACHE`.



PROBLEMA · 05

Não monitorar sys.dm_exec_procedure_stats

SP em produção por anos sem ninguém olhando métricas métricas — param sniffing só é descoberto quando vira vira incidente.

CORREÇÃO

Dashboard diário com top 10 SPs por `avg_elapsed_time` e `execution_count`. Configurar alertas em Extended Events para regressões.



PROBLEMA · 06

SPs com lógica de negócio espalhada

Dev coloca validações, cálculos e fluxo de negócio dentro dentro da SP — vira 'aplicação em T-SQL', difícil de testar e manter.

CORREÇÃO

SPs devem ser thin wrappers sobre CRUD/atOMICIDADE. Lógica de domínio fica na aplicação (testável, versionável). Use SP para: transações ACID, validações de banco, performance crítica.

✓ Padrão ouro: monitorar + parametrizar + conceder EXECUTE mínimo

◆ DESAFIO · TÓPICO 2.2

Desafio: Diagnosticar e Mitigar Parameter Sniffing

Cenário realista — você é o DBA responsável pela solução.

ENUNCIADO

Você é DBA de um e-commerce. A SP `dbo.usp_BuscaProdutos` recebe `@CategoriaId INT` e `@PrecoMax @PrecoMax DECIMAL(10,2)`. Em horário comercial, rodando para `CategoriaId = 5` (10 produtos) a SP demora 50ms. Após o horário comercial, rodando para `CategoriaId = 1` (100k produtos) a SP demora 8s. Após esse evento, todas as chamadas para `CategoriaId = 5` passam a demorar 8s — só resolve reiniciando o

TAREFAS

- 01 Confirmar o diagnóstico de parameter sniffing (cite as DMVs/queries que executaria e o que procuraria).
- 02 Propor a versão otimizada da SP, justificando a técnica escolhida (OPTIMIZE FOR vs RECOMPILE vs variável local).
- 03 Adicionar tratamento de segurança para evitar que a SP seja chamada com parâmetros SQL Injection via SQL Injection via SQL dinâmico (se aplicável).
- 04 Especificar uma Extended Events session para capturar regressões futuras.

DICAS

- Comece por `sys.dm_exec_procedure_stats` + `sys.dm_exec_query_plan` para ver o plano cached.
- Compare com plano estimado via `SET SHOWPLAN_XML ON` rodando para `CategoriaId = 5` sem cache.
- Se planos diferem, confirmado param sniffing. Caso típico → OPTIMIZE FOR UNKNOWN; caso extremo → RECOMPILE.
- XEvents: capturar `sp_statement_completed` + `sql_text` filtrando por `object_name = 'usp_BuscaProdutos'`.

Critérios de Avaliação

CHECKLIST · 8 ITENS

- ✓ 01 Diagnóstico correto usando as DMVs certas
- ✓ 02 Identificação do plano cached problemático
- ✓ 03 Escolha de mitigação justificada com trade-offs
- ✓ 04 Aplicação de OPTIMIZE FOR ou RECOMPILE
- ✓ 05 `sp_executesql` com parâmetros (anti-injection)
- ✓ 06 XEvents session com filtros corretos
- ✓ 07 Estimativa de impacto em produção
- ✓ 08 Documentação das mudanças

SÍNTESE DO MÓDULO

O que vimos hoje

01

TÓPICO 2.1

Criação, Execução e Parametrização

Anatomia e ciclo de vida de SPs (compile→cache→execute→reuse), opções de criação (WITH (WITH ENCRYPTION, RECOMPILE, EXECUTE AS), parametrização estrita (Input, OUTPUT, TVP), OUTPUT, TVP), tratamento de erros moderno (TRY/CATCH + THROW + XACT_STATE) e anti-patterns anti-patterns comuns.

02

TÓPICO 2.2

Otimização, Segurança e Monitoramento

Parameter Sniffing (causa, efeito e 3 mitigações: OPTIMIZE FOR, RECOMPILE, variável local), reutilização do Plano de Execução (Plan Cache), Ownership Chaining + permissão EXECUTE, sp_executesql parametrizado (anti-SQL-injection), e monitoramento via DMVs (sys.dm_exec_procedure_stats) + Extended Events.

→ Próximos Passos

- Reproduzir todos os códigos no SSMS local (SQL Server Developer Edition).
- Implementar o desafio do slide 11 (SP de Registro de Pedidos com TVP).
- Diagnosticar uma SP lenta em produção usando sys.dm_exec_procedure_stats.
- Aplicar OPTIMIZE FOR UNKNOWN em SPs com catch-all (@param IS NULL OR IS NULL OR ...).
- Auditar permissões: verificar SELECTs diretos concedidos a apps — substituir por EXECUTE.

 proxima_aula.sql

SQL

```
-- Próxima aula: Módulo 3 — Triggers
-- DML, DDL e LOGON
SELECT TOP 5
FROM
ORDER BY      DESC
```

Módulo 3: Triggers — DML, DDL, LOGON, anti-patterns e uso responsável.